

eg.

- `assertTrue(x.compareTo(x) == 0);` // `x==x`
- `assertTrue(x.compareTo(y) != 0);` // `x!=y`
- `assertTrue(xy.compareTo(xy) == 0);` // `xy==xy`
- `assertTrue(x.compareTo(y) == -1);` // `x<y`
- `assertTrue(xx.compareTo(xy) == -1);` // `xx<xy`
- `assertTrue(yx.compareTo(yy) == -1);` // `yx<yy`
- `assertTrue(yy.compareTo(yx) == 1);` // `yy>yx`

• **TODO comments**

It is sometimes reasonable to leave “To do” notes in the form of `//TODO` comments. In the following case, the `TODO` comment explains why the function has a degenerate implementation and what that function’s future should be.

- `//TODO-MdM these are not needed`
 - `// We expect this to go away when we do the checkout`
- model
- `protected VersionInfo makeVersion() throws Exception`
 - `{`
 - `return null;`
 - `}`

• **Javadocs**

While writing public APIs you should definitely write good javadocs. They are extremely helpful

eg. Javadocs for the Java Standard Library

Bad comments

Unfortunately, the comments that many programmers write fall in this category.

• **Ambiguous / unclear comments**

Writing a comment just for the sake of writing one, leads to unclear and ambiguous comments. If you do decide to write a comment, spend time and make sure you write it in the simplest and clearest way.

• **Redundant comments**

Comments are meant to express what we failed to express via code. Their purpose is not to translate code to english.